LAPORAN

PRAKTIKUM PEMROGRAMAN PERANGKAT MOBILE

Sesi 03: implementasi state management (provider)studi kasus

Aplikasi NusaBookstore

Disusun Oleh :

Nama : Moch.Arief Rifansyah
Nim : 20240040194
Kelas : TI24B

Dosen Pengampu:

YULHAN WAHYUDIN, M.Kom

PROGRAM STUDI TEKNIK INFORMATIKA

UNIVERSITAS NUSA PUTRA

DAFTAR ISI

DAFTAR ISI i
BAB I PENDAHULUAN & VERIFIKASI LINGKUNGAN 1
BAB II STRUKTUR PROYEK & ANALISIS KODE 2
1. Lapisan Model (lib/models/) 2
2. Titik Masuk (Entry Point) & Reaktivitas 2
BAB III ANALISIS KOMPARASI TEKNIS: PROVIDER VS SETSTATE 3
1. Pendekatan Tradisional menggunakan setState() 3
2. Pendekatan Modern menggunakan Provider 3
BAB IV HASIL UJI COBA APLIKASI 4
1. Tampilan Katalog (Light Mode & Dark Mode) 4
2. Sinkronisasi Keranjang Belanja 5
BAB V TROUBLESHOOTING LOG 7
1. Kendala: Kegagalan Render Gambar Sampul Buku (Network Image) 7
2. Kendala: Error Package 'intl' (Undefined name 'NumberFormat') 7

BAB I
PENDAHULUAN & VERIFIKASI LINGKUNGAN

Praktikum kali ini berfokus pada implementasi *State Management* modern pada aplikasi *e-commerce* buku (NusaBookstore) menggunakan ekosistem Flutter. Pendekatan yang digunakan adalah arsitektur Provider, yang secara resmi direkomendasikan oleh tim Flutter untuk menggantikan manajemen status bawaan yang kurang efisien pada skala aplikasi menengah hingga besar.

Tujuan utama dari pengembangan ini adalah memisahkan antara antarmuka pengguna (UI) dan logika bisnis, mengelola status keranjang belanja (*cart*) secara *real-time*, serta menerapkan pergantian tema dinamis (*light/dark mode*). Sebelum pengembangan dimulai, lingkungan kerja (*environment*) di Visual Studio Code telah dipastikan siap, termasuk pembaruan *package* melalui file pubspec.yaml untuk mengunduh dependensi seperti provider dan intl (untuk pemformatan mata uang).

BAB II
STRUKTUR PROYEK & ANALISIS KODE

Untuk menghindari penumpukan logika bisnis di dalam satu berkas antarmuka (UI), proyek ini mengadopsi prinsip *Clean Code* dengan merancang struktur modular. Direktori utama lib/ dipisahkan menjadi tiga lapisan fungsional:

1. **Lapisan Model (lib/models/)**

- **Lapisan Model (lib/models/):** Menampung berkas book.dart yang mendefinisikan kerangka data (struktur *blueprint*) untuk entitas buku.
- **Lapisan Provider (lib/providers/):** Tempat isolasi logika bisnis. cart_provider.dart menangani fungsi penambahan buku dan kalkulasi total harga, sedangkan theme_provider.dart mengatur palet warna aplikasi secara global.
- **Lapisan Antarmuka (lib/screens/):** Berisi catalog_screen.dart dan cart_screen.dart yang murni bertugas merender komponen visual tanpa memuat kalkulasi matematika rumit.

2. **Titik Masuk (Entry Point) & Reaktivitas**

Berkas main.dart difokuskan sebagai titik konfigurasi utama di mana MultiProvider didaftarkan pada level teratas (runApp). Ketika pengguna berinteraksi (misalnya menekan tombol "Tambah"), fungsi addItem() dipanggil dan notifyListeners() akan merender ulang bagian spesifik yang mendengarkan perubahan (seperti *badge* notifikasi keranjang) tanpa harus memuat ulang seluruh halaman UI.

BAB III
ANALISIS KOMPARASI TEKNIS: PROVIDER VS SETSTATE

Dalam pengembangan aplikasi Flutter, manajemen status adalah komponen krusial untuk memastikan antarmuka pengguna selalu sinkron dengan data di balik layar. Terdapat perbedaan arsitektur yang sangat signifikan antara penggunaan setState() bawaan dengan arsitektur Provider:

1. Pendekatan Tradisional menggunakan setState()

Jika aplikasi NusaBookstore ini hanya mengandalkan setState(), manajemen data keranjang belanja akan menjadi sangat kompleks. Data harus dioper secara manual antar halaman melalui konstruktor.

- **Kelemahan (Prop Drilling):** Proses mengoper data secara berantai ini membuat kode saling ketergantungan (tight coupling). Selain itu, setiap penambahan buku ke keranjang akan memicu setState() untuk melakukan render ulang (rebuild) pada keseluruhan halaman, yang memboroskan memori dan menurunkan performa aplikasi.

2. Pendekatan Modern menggunakan Provider

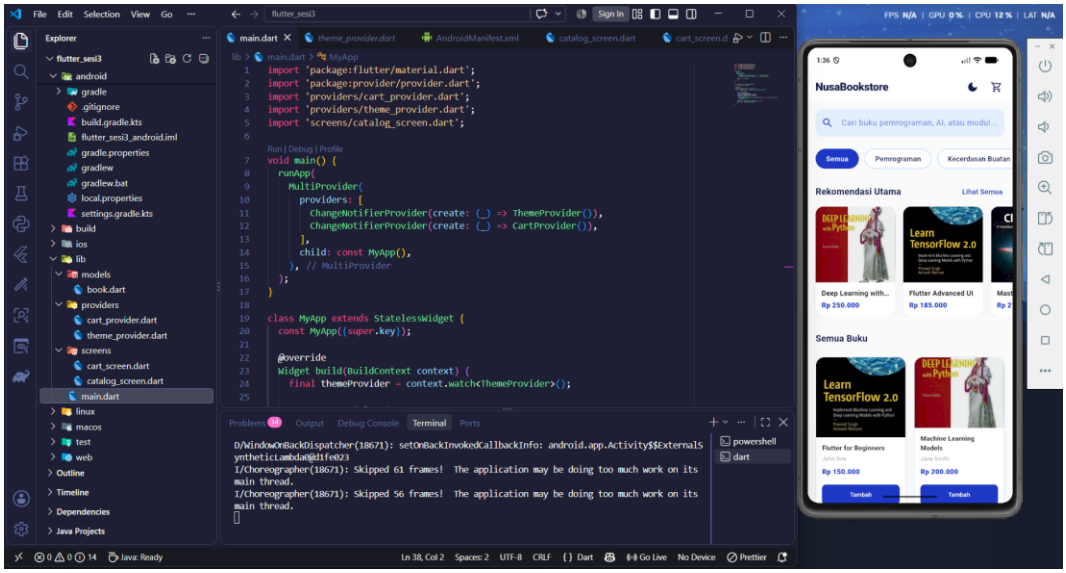
Dengan mengimplementasikan MultiProvider, data keranjang dan tema diangkat menjadi State Global.

- **Keunggulan Arsitektur:** Pemisahan struktur sangat jelas dan terhindar dari Prop Drilling. Flutter HANYA melakukan render ulang pada bagian spesifik yang mendengarkan perubahan data, menjadikan aplikasi jauh lebih efisien, ringan, dan siap untuk dikembangkan pada skala yang lebih besar (scalable).

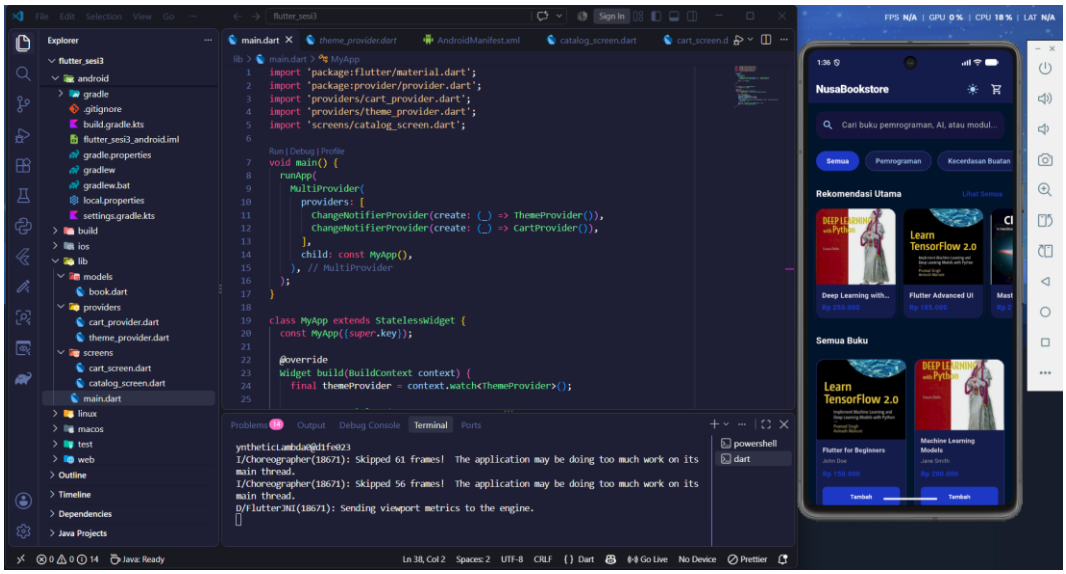
BAB IV
HASIL UJI COBA APLIKASI

Pengujian fungsional dilakukan pada Emulator Android untuk memastikan seluruh reaktivitas berjalan sesuai skenario arsitektur yang dibangun.

1. Tampilan Katalog (Light Mode & Dark Mode)



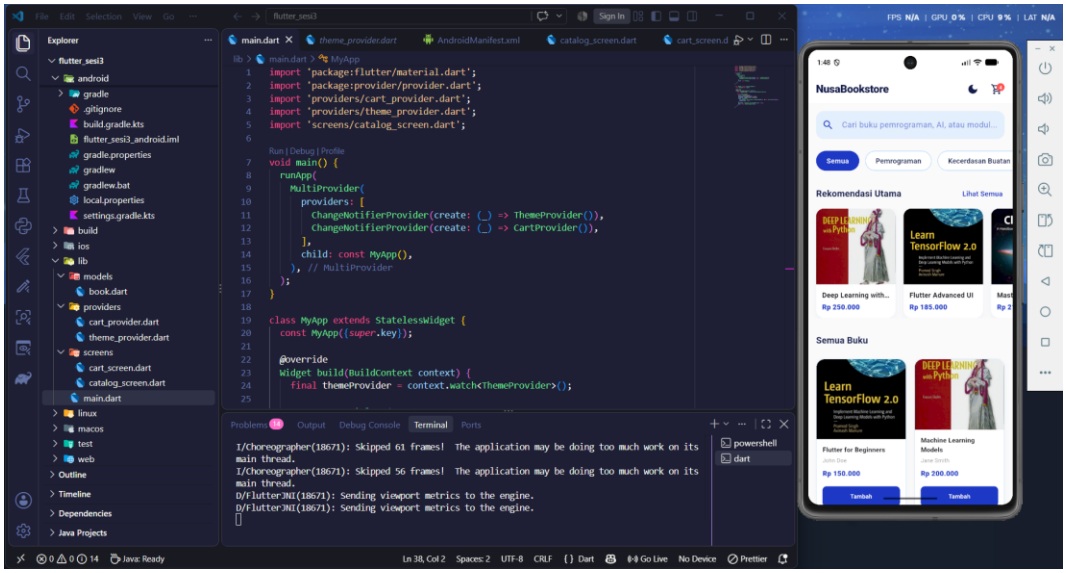
Gambaar 4 .1 *Tampilan Halaman Utama (Katalog) dalam Light Mode.*



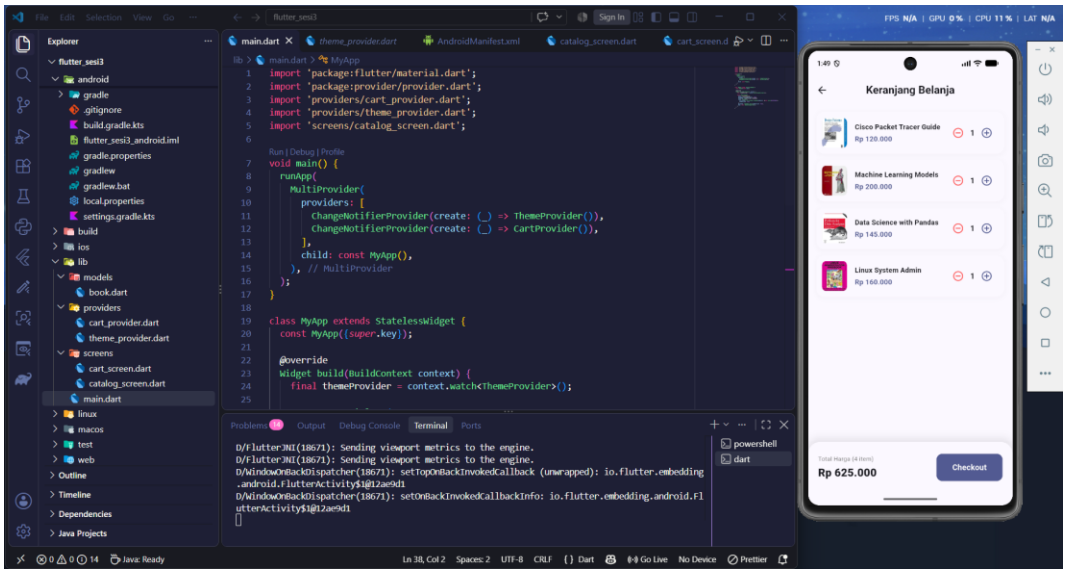
Gambaar 4 .2 *Tampilan Halaman Utama dalam Dark Mode sebagai respons dari ThemeProvider.*

Analisis: Pergantian tema menggunakan ThemeProvider merespons secara sempurna. Seluruh elemen UI menyesuaikan kontras warnanya secara instan tanpa mengganggu status keranjang belanja.

2. Sinkronisasi Keranjang Belanja



Gambaar 4 .3 *Indikator Badge Keranjang (Merah) yang terbarui secara real-time saat buku ditambahkan.*



Gambaar 4 .4 *Halaman Cart Screen menampilkan detail item belanja dan kalkulasi total harga dalam format Rupiah*

Analisis: Penambahan item memicu pembaruan badge keranjang secara real-time. Di halaman CartScreen, perhitungan kuantitas dan format harga menggunakan package intl (Rupiah) beroperasi secara akurat sesuai fungsi kalkulasi pada CartProvider.

BAB V
TROUBLESHOOTING LOG

Selama proses pengembangan dan implementasi State Management, terdapat beberapa kendala teknis yang didokumentasikan dan diselesaikan sebagai berikut:

1. **Kendala: Kegagalan Render Gambar Sampul Buku (Network Image)**

- **Gejala:** Saat aplikasi pertama kali dijalankan, gambar sampul tidak muncul (ditandai dengan error 403 Forbidden pada log terminal) dan emulator belum memiliki izin akses internet secara native.
- **Solusi:** URL gambar dikonfigurasi ulang menggunakan API terbuka dari OpenLibrary. Selanjutnya, dilakukan modifikasi pada sistem Android dengan menyuntikkan kode <useer permission android: name=”android.permisssion.INTERNET”> Kedalam AndroidManifest.xml. Setelah aplikasi di- Cold Boot, gambar berhasil direnderdengan sempurna.

2. **Kendala: Error Package 'intl' (Undefined name 'NumberFormat')**

- **Gejala:** Pada saat implementasi format mata uang Rupiah (Rp), IDE memunculkan peringatan Target of URI doesn't exist untuk package intl, sehingga aplikasi gagal dikompilasi.

- **Solusi:** Memastikan path terminal berada pada direktori root proyek, kemudian menjalankan perintah flutter pub add intl untuk mendaftarkan dependensi ke dalam pubspec.yaml. Setelah tersinkronisasi, error teratasi dan format mata uang berjalan normal.